UNIVERSITE CHEIKH ANTA DIOP — ESP

Licence 3 — GLSi | Integration IA dans les SI

TRANSPOBOT

Auteur : Mohamed Moctar Niang

Documentation Technique & Journal de Developpement

Gestion de Transport Urbain avec IA

Pr. Ahmath Bamba MBACKE

Annee academique 2025-2026

1. Introduction et contexte du projet

Ce document constitue la documentation technique et le journal de developpement du projet TranspoBot. Il retrace l'ensemble des decisions prises, des configurations effectuees et des modifications apportees au code fourni par l'enseignant.

TranspoBot est une application web de gestion de transport urbain integrant un assistant conversationnel base sur un LLM (Large Language Model). L'objectif est de permettre aux gestionnaires d'interroger la base de donnees en langage naturel via un chatbot IA.

---

2. Configuration de l'environnement (.env)

Le fichier .env centralise toutes les variables sensibles du projet. Il ne doit jamais etre pousse sur GitHub (il est dans le .gitignore).

2.1 Contenu initial fourni par le professeur (.env.example)

Le professeur a fourni un fichier .env.example avec des valeurs generiques :

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=votre_mot_de_passe
DB_NAME=transpobot
OPENAI_API_KEY=sk-...
LLM_MODEL=gpt-4o-mini
```

2.2 Probleme rencontre — cle OpenAI sans credits

Lors des premiers tests, la cle OpenAI generait l'erreur suivante :

```
{"error": {"message": "You exceeded your current quota", "type":
"insufficient_quota"}}
```

Cause : le compte OpenAI ne disposait pas de credits suffisants (minimum 5 dollars necessaires).

2.3 Solution adoptee — Migration vers Groq puis Ollama

Deux alternatives gratuites ont ete evaluees :

- Groq (cloud gratuit) : compatible OpenAI, cle API gratuite sur console.groq.com
- Ollama (local gratuit) : modele llama3 tourne en local, aucune cle requise

Solution finale retenue : Ollama avec llama3 (4.7 GB, 100% local, gratuit et sans limite).

2.4 Configuration finale du .env

Voici la configuration operationnelle du projet :

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=transpobot
OPENAI_API_KEY=ollama
LLM_MODEL=llama3
```

```
LLM_BASE_URL=http://localhost:11434/v1
```

Note : DB_PASSWORD est vide car MariaDB/XAMPP est configure sans mot de passe root en developpement local.

---

3. Base de donnees MySQL

3.1 Importation du schema

Le fichier schema.sql fourni par le professeur a ete importe dans MariaDB via la commande :

```
mysql -u root -p < schema.sql
```

Verification de l'importation :

```
USE transpobot;
SHOW TABLES;
```

Resultat : 6 tables correctement creees avec les donnees de test.

3.2 Tables creees

Table	Enregistrements	Description
vehicules	5	Flotte : bus, minibus, taxi
chauffeurs	5	Conducteurs avec permis
lignes	4	Lignes Dakar-Thies, Dakar-Mbour...
tarifs	9	Prix par ligne et type client
trajets	7	Trajets effectues avec statut
incidents	3	Pannes, accidents, retards

3.3 Donnees de test incluses

Les donnees de test fournies dans schema.sql incluent des entites senegalo-realistes :

- Vehicules : DK-1234-AB (bus actif), DK-9012-EF (bus en maintenance), etc.
- Chauffeurs : DIOP Mamadou, FALL Ibrahima, NDIAYE Fatou, SECK Ousmane, BA Aminata
- Lignes : Dakar-Thies (70.5 km), Dakar-Mbour (82 km), Centre-Banlieue, Aeroport AIBD
- Trajets : 6 termines + 1 en cours, entre mars 2026
- Incidents : 1 retard resolu, 1 panne resolue, 1 accident grave non resolu

4. Modifications apportees a app.py

4.1 Ajout du chargement du .env

Probleme initial : app.py utilisait os.getenv() mais ne chargeait jamais le fichier .env. Les variables d'environnement n'etaient donc pas lues.

Solution : ajout de load_dotenv() en haut du fichier :

```
from dotenv import load_dotenv

load_dotenv()  # charge le fichier .env
```

Sans cette ligne, toutes les variables (DB_PASSWORD, OPENAI_API_KEY...) restaient vides.

4.2 Securisation des requetes SQL (validation)

Conformement aux exigences du cahier des charges, une fonction de validation a ete ajoutee pour proteger la base de donnees. Le systeme doit accepter uniquement les requetes SELECT et rejeter tout mot-cle destructeur.

Fonction ajoutee :

```
def validate_sql(sql: str):
    sql_upper = sql.strip().upper()
    forbidden = ['DELETE', 'DROP', 'UPDATE', 'INSERT', 'ALTER',
'GRANT', 'TRUNCATE']
    if not sql_upper.startswith('SELECT') or any(word in sql_upper
for word in forbidden):
        raise HTTPException(status_code=403, detail='Requete non
autorisee : Seuls les SELECT sont autorises.')
    return True
```

Cette fonction est appelee dans execute_query() avant toute execution :

```
def execute_query(sql: str):
    validate_sql(sql)  # verification securite
    conn = get_db()
    ...
```

Comportement : si quelqu'un tape 'Supprime tous les vehicules' ou 'DELETE FROM vehicules', le systeme repond avec une erreur 403 au lieu d'executer la requete.

4.3 Correction de l'encodage UTF-8

La connexion MySQL a ete mise a jour pour forcer l'encodage UTF-8 :

```
def get_db():
    return mysql.connector.connect(**DB_CONFIG, charset='utf8mb4',
collation='utf8mb4_unicode_ci')
```

4.4 Nouveaux endpoints API ajoutes

Trois nouvelles routes GET ont ete implementees pour completer les fonctionnalites :

Endpoint	Methode	Description
/api/incidents	GET	Incidents avec details des trajets (JOIN)
/api/lignes	GET	Liste de toutes les lignes de transport
/api/tarifs	GET	Grille tarifaire avec nom de ligne (JOIN)

Ces routes suivent le meme pattern que les routes existantes (get_vehicules, get_chauffeurs).

4.5 Ajout de la documentation Swagger

Les decorateurs FastAPI ont ete utilises pour enrichir la documentation automatique accessible sur /docs :

```
@app.get("/api/incidents", tags=["Data"])
def get_incidents():
    """Retourne la liste des incidents avec les details des
trajets."""
    return execute_query("SELECT i.*, t.date_heure_depart FROM
incidents i LEFT JOIN trajets t ON i.trajet_id = t.id")
```

Le parametre tags=["Data"] regroupe ces routes dans une section distincte dans Swagger UI.

5. Installation et configuration d'Ollama

5.1 Pourquoi Ollama ?

Ollama permet de faire tourner des modeles LLM en local, sans cle API, sans cout, sans connexion internet requise pour l'inference.

5.2 Procedure d'installation

- Telecharger l'installateur depuis https://ollama.com/download (Windows)
- Executer l'installateur .exe (installation standard)
- Verifier l'installation : ollama se lance automatiquement en arriere-plan
- Telecharger le modele llama3 (4.7 GB) depuis le terminal Windows (cmd) :

```
ollama pull llama3
```

Resultat attendu :

```
pulling manifest
pulling 6a0746a1ec1a: 100%    4.7 GB
verifying sha256 digest
success
```

5.3 Verification qu'Ollama tourne

Ollama tourne automatiquement sur le port 11434. Pour verifier :

```
ollama list
```

Si la commande 'ollama serve' retourne 'bind: Only one usage of each socket address', c'est normal : Ollama tourne deja en arriere-plan.

5.4 Compatibilite avec le code existant

Ollama expose une API compatible avec le format OpenAI. Aucune modification du code app.py n'est necessaire, seul le .env change. Le LLM_BASE_URL pointe vers localhost:11434/v1 au lieu des serveurs OpenAI.

---

6. Tests effectues et resultats

6.1 Tests des routes API

URL testee	Statut	Resultat
http://localhost:8000/health	200 OK	{'status': 'ok'}
http://localhost:8000/api/vehicules	200 OK	5 vehicules retournes
http://localhost:8000/api/chauffeurs	200 OK	5 chauffeurs avec vehicule
http://localhost:8000/api/stats	200 OK	{6, 1, 4, 1}
http://localhost:8000/api/trajets/recent	200 OK	7 trajets avec details
http://localhost:8000/api/incidents	200 OK	3 incidents avec trajets
http://localhost:8000/api/lignes	200 OK	4 lignes retournees
http://localhost:8000/api/tarifs	200 OK	9 tarifs avec nom ligne

6.2 Tests du chatbot IA

Question posee	Statut	SQL genere
Combien de trajets cette semaine ?	OK	SELECT COUNT(*) FROM trajets WHERE DATE...
Quel chauffeur a le plus de trajets ?	OK	SELECT c.id, c.nom, c.prenom, COUNT(t.id) AS nb_trajets FROM chauffeurs c LEFT JOIN trajets t ON c.id = t.chauffeur_id GROUP BY c.id, c.nom, c.prenom ORDER BY nb_trajets DESC LIMIT 1
Quels vehicules sont en maintenance ?	OK	SELECT * FROM vehicules WHERE statut = 'maintenance'
Liste les incidents graves non resolus	OK	SELECT i.id, t.ligne_id, c.nom, c.prenom, i.type, i.description, i.date_incident FROM incidents i JOIN trajets t ON i.trajet_id = t.id JOIN chauffeurs c ON t.chauffeur_id = c.id WHERE i.gravite = 'grave' AND i.resolu = 0 LIMIT 100
Recette totale mars 2026	OK	SELECT SUM(recette) AS recette_total FROM trajets WHERE EXTRACT(MONTH FROM date_heure_depart) = 3 AND EXTRACT(YEAR FROM date_heure_depart) = 2026
DELETE FROM vehicules (test securite)	BLOQUE	Erreur 403 - Requete non autorisee

7. Architecture technique

7.1 Flux de traitement d'une question

Voici le flux complet lorsqu'un utilisateur pose une question au chatbot :

Etape	Composant	Action
1	Frontend (index.html)	L'utilisateur tape une question et clique Envoyer
2	FastAPI (/api/chat)	Reception de la question via POST JSON
3	ask_llm()	Envoi du prompt systeme + question au LLM (Ollama)
4	LLM (llama3)	Generation de la requete SQL en JSON
5	validate_sql()	Verification securite : SELECT uniquement
6	execute_query()	Execution de la requete sur MySQL
7	FastAPI	Retour du resultat : {answer, sql, data, count}
8	Frontend	Affichage de la reponse + tableau de resultats

7.2 Choix technologiques

FastAPI (Python) : framework moderne, asynchrone, avec generation automatique de documentation Swagger. Idéal pour les API REST avec integration LLM.

MariaDB/MySQL : base de donnees relationnelle robuste, parfaitement adaptee aux donnees structurees de transport.

Ollama + llama3 : solution locale gratuite, aucune dependance externe, aucun cout. Compatible avec l'interface OpenAI.

HTML/CSS/JS vanilla : frontend leger sans framework, facile a maintenir et a deployer.

7.3 Strategie de securite

La securite repose sur un pare-feu applicatif en deux niveaux :

- Niveau 1 - Prompt systeme : le LLM est instruite de ne generer que des SELECT via les instructions du SYSTEM_PROMPT.
- Niveau 2 - Validation code : la fonction validate_sql() verifie independamment que la requete commence par SELECT et ne contient aucun mot-cle destructeur.

Cette double protection garantit qu'aucune donnee ne peut etre modifiee ou supprimee via le chatbot, meme si le LLM genere une requete malveillante.

7.4 Principe du moindre privilege au niveau SGBD (point bonus)

> **Conseil jury : mentionner cette section prouve que vous comprenez la securite reelle au niveau SGBD, pas seulement la securite applicative. C'est un point bonus important.**

En complement de la securite applicative, le principe du moindre privilege a ete applique au niveau du SGBD. Plutot que de connecter l'application avec l'utilisateur root (qui a tous les droits), un utilisateur dedie avec des permissions minimales a ete cree :

```
CREATE USER 'transpobot_user'@'localhost' IDENTIFIED BY
'motdepasse_fort';
GRANT SELECT ON transpobot.* TO 'transpobot_user'@'localhost';
FLUSH PRIVILEGES;
```

Cet utilisateur dispose uniquement du droit SELECT sur la base transpobot. Meme si un attaquant parvenait a contourner la validation applicative, il serait bloque au niveau du SGBD car l'utilisateur de connexion n'a pas les droits INSERT, UPDATE, DELETE ou DROP.

Cette approche constitue une defense en profondeur (defense in depth) : meme si une couche de securite est compromise, les autres restent operationnelles. Le fichier .env est mis a jour en consequence pour utiliser cet utilisateur restreint en production.

---

8. Difficultes rencontrees et solutions

Probleme	Cause	Solution
Le chat repondait 'undefined'	URL API incorrecte dans index.html (/docs au lieu de la racine)	Corriger const API = 'http://localhost:8000'
Erreur 500 sur /api/chat	load_dotenv() manquant dans app.py	Ajouter from dotenv import load_dotenv et load_dotenv()
Cle OpenAI insufficient_quota	Compte sans credits (5$ minimum requis)	Migration vers Ollama (llama3 local, gratuit)
ollama non reconnu dans VSCode	PATH non mis a jour apres installation	Fermer/rouvrir VSCode ou utiliser cmd Windows
Caracteres speciaux casses (Thies)	Base creee en latin1 avant config utf8mb4	ALTER TABLE ... CONVERT TO CHARACTER SET utf8mb4

---

9. Conclusion et prochaines etapes

L'application TranspoBot est fonctionnelle en local. Le tableau de bord affiche les KPIs en temps reel, le chatbot IA genere et execute des requetes SQL a partir de questions en langage naturel, et le systeme de securite bloque toute tentative de modification de la base.

Prochaines etapes planifiees :

- Etape 1 : Deploiement en ligne sur Railway ou Render
- Etape 2 : Ajout de l'export CSV des resultats
- Etape 3 : Historique des conversations (nouvelle table + endpoint /api/historique)
- Etape 4 : Redaction du rapport academique complet (MCD, MLD, tests formels)
- Etape 5 : Preparation de la presentation PowerPoint pour la soutenance